Overriding, Polymorphism, Constructors & Abstract Classes

Formatted by: Heisenberg 😎

Object-Oriented Programming — Week 5 (Java)

Today's session picks up the story of an object's *life* — how it is born (constructors), how it behaves (methods), what it shares with its siblings (static members), how it refers to itself (`this`), how it is organized into neighborhoods (packages and access modifiers), how it protects its secrets (encapsulation), how it inherits family traits (inheritance), how it wears different masks (polymorphism), how it hides its machinery (abstraction), and how it still respects its parents (`super`).

Index

Section	What you'll actually find there (with a wink)
1. Constructors	The mysterious method that runs itself — no invitation needed.
2. Constructor Overloading	One class, many birth certificates.
3. Methods	Blocks of code with a job description.
4. Method Overloading	Same name, different costumes, same job title.
5. Static Members	The family heirloom every object shares.
6. The `this` Keyword	An object pointing at itself and saying "me!"
7. Packages & Access Modifiers	Java's gated communities and their guest lists.
8. Encapsulation	Locking the diary, handing out a key only you control.
9. Inheritance	The family tree, and who inherits what.
10. Polymorphism	One name, many faces — compile-time and run-time.
11. Abstraction	Showing the steering wheel, hiding the engine.
12. The `super` Keyword	Calling your parents before you do your own thing.
13. Summary	The whole story in five bullet points.
14. Practice Questions	Homework that actually makes you think.

---

1. Constructors

What is a Constructor?

A **constructor** is a special member function used to **initialize an object**. Think of it as the "birth certificate generator" of a class — the moment an object is born (created with `new`), the constructor runs automatically to set it up.

Characteristics:

- Has the **same name as the class**.
- Has **no return type** — not even `void` . Compare this to a normal method like `main` , which has `void` , or `add` , which might return an `int` . A constructor breaks that pattern entirely.
- It is **invoked automatically** the instant an object is created — you never call it yourself with a dot, the way you would call `s.display()` .

Default Constructor

A constructor with **no arguments** is called a default constructor. It runs automatically when the object is created, purely to set up the object.

```
class Student {

  Student() {
    System.out.println("Object Created");
  }

  public static void main(String args[]) {
    Student s = new Student();
  }
}
```

Output

```
Object Created
```

The moment `new Student()` executes, Java quietly calls the constructor behind the scenes — you never see `s.Student()` written anywhere, yet it runs.

Parameterized Constructor

A **parameterized constructor** accepts one or more arguments so that you can initialize the instance variables (fields) of an object with **specific, custom values** at the exact moment it is created.

Unlike a default constructor — which sets fields to blank or generic values — a parameterized constructor hands you **full control** over the initial state of your object.

```
class Student {
  int id;
  String name;

  Student(int i, String n) {
    id = i;
    name = n;
  }

  void display() {
    System.out.println(id + " " + name);
  }

  public static void main(String args[]) {
    Student s = new Student(101, "Rahul");
    s.display();
  }
}
```

Output

```
101 Rahul
```

Here `101` flows into `i`, and `"Rahul"` flows into `n`; inside the constructor these get copied into the fields `id` and `name`. The object is now "born" already knowing who it is.

> **Note for Review:** A very common exam trap is assuming a class *must* have a default constructor. The moment you write **any** parameterized constructor yourself, Java stops providing the free default one — if you still want `new Student()` to work, you must write that constructor explicitly.

2. Constructor Overloading

Constructor overloading means having **multiple constructors** in the same class, each with a **different parameter list** — no arguments, one argument, two arguments, three arguments, and so on. Java picks the right one to run based on how many (and what type of) arguments you pass when you write `new`.

```
class Rectangle {
  int length;
  int width;

  Rectangle() {
    length = 1;
    width = 1;
  }

  Rectangle(int l, int w) {
    length = l;
    width = w;
  }

  void area() {
    System.out.println(length * width);
  }

  public static void main(String args[]) {
    Rectangle r1 = new Rectangle();
    Rectangle r2 = new Rectangle(5, 4);

    r1.area();
    r2.area();
  }
}
```

Output

```
1
20
```

`r1` is created with the zero-argument constructor, so `length = width = 1`, giving an area of `1`. `r2` is created by matching the two-argument constructor, so `length = 5`, `width = 4`, giving an area of `20`. This is the essence of overloading: **one class, several "birth plans,"** each doing initialization differently depending on what information you hand over.

3. Methods

What is a Method?

A **method** is a block of code that performs a specific task — think of it as a single employee in your program, each hired to do one job (add, subtract, print, and so on).

Syntax

```
returnType methodName(parameters) {
   // body
}
```

- `returnType` can be `int`, `double`, `String`, `void`, etc.
- `methodName` is written in small (camelCase) letters — never capitalized like a class name.
- `parameters` may be present or completely absent, depending on what the method needs to do its job.

Example

```
class Calculator {

  int add(int a, int b) {
    return a + b;
  }

  public static void main(String args[]) {
    Calculator c = new Calculator();
    System.out.println(c.add(5, 3));
  }
}
```

Output

```
8
```

Notice you must first **create an object** (`c = new Calculator()`) before you can call a non-static method — a method cannot be summoned out of thin air; it needs an object to belong to.

---

4. Method Overloading

Method Overloading means having methods with the **same name but different parameter lists** — different number of arguments, and/or different argument types. This is exactly the same idea as constructor overloading, except the name doesn't have to match the class name — it can be `add` , `subtract` , or anything you like.

```
class Calculator {

  int add(int a, int b) {
    return a + b;
  }

  double add(double a, double b) {
    return a + b;
  }

  int add(int a, int b, int c) {
    return a + b + c;
  }

  public static void main(String args[]) {
    Calculator c = new Calculator();

    System.out.println(c.add(5, 4));
    System.out.println(c.add(2.5, 3.5));
    System.out.println(c.add(1, 2, 3));
  }
}
```

Output

```
9
6.0
6
```

Java looks at *how many* arguments you passed and *what type* they are, then matches the call to the correct version of `add` — this decision is made entirely **at compile time**, which is why method overloading is also called **compile-time polymorphism** (more on that in Section 10).

> **Note for Review:** Constructor overloading vs. Method overloading — don't mix them up! A constructor's name is *forced* to match the class name; a method's name is free to be anything. Both share the same underlying idea: same name, different signatures.

5. Static Members

Static Variable

A **static variable** is shared by **all objects** of a class — instead of each object keeping its own private copy, there is exactly **one copy** that everyone points to.

```
class Student {
  int roll;
  String name;

  static String college = "IIT Jodhpur";

  void display() {
    System.out.println(roll + " " + name + " " + college);
  }

  public static void main(String args[]) {
    Student s1 = new Student();
    Student s2 = new Student();

    s1.roll = 1;
    s1.name = "Amit";

    s2.roll = 2;
    s2.name = "Neha";

    s1.display();
    s2.display();
  }
}
```

Output

```
1 Amit IIT Jodhpur
2 Neha IIT Jodhpur
```

`roll` and `name` are per-object (each student has their own), but `college` is **one shared value** — both `s1` and `s2` display the exact same `"IIT Jodhpur"`, because there is only a single static copy living at the class level, not inside each object.

Static Method

A **static method** belongs to the **class itself**, not to any individual object — so you can call it without ever creating an object.

```
class MathDemo {

  static int square(int x) {
    return x * x;
  }

  public static void main(String args[]) {
    System.out.println(MathDemo.square(5));
  }
}
```

Output

```
25
```

You call it as `MathDemo.square(5)` — directly through the class name, no `new` required.

Static Block

A **static block** executes **only once**, the very moment the class is loaded into memory — even before `main` runs.

```
class Demo {

  static {
    System.out.println("Static Block Executed");
  }

  public static void main(String args[]) {
    System.out.println("Main Method");
  }
}
```

Output

```
Static Block Executed
Main Method
```

Even though `main` is the entry point of the program, the static block **jumps the queue** and executes first, because loading the class itself triggers the block, before Java even starts executing statements inside `main`.

> **Note for Review:** Order of execution at class-loading time: **static blocks** → **static variables' initializers** → **then** `main`. This ordering question shows up often in exams.

---

6. The `this` Keyword

The `this` keyword refers to the **current object** — the specific object on which a method or constructor is currently operating.

Resolving Variable Name Conflict

```
class Student {
  int id;

  Student(int id) {
    this.id = id;
  }

  void display() {
    System.out.println(id);
  }

  public static void main(String args[]) {
    Student s = new Student(101);
    s.display();
  }
}
```

Output

```
101
```

Here the constructor's parameter is also named `id` — same name as the instance variable. Without `this`, Java would get confused about which `id` you mean. Writing `this.id = id;` clearly says: *"the field belonging to this current object gets the value of the parameter."*

Calling Another Constructor

The `this()` call (note the parentheses) lets one constructor invoke **another constructor** in the same class.

```
class Student {
  Student() {
    this(100);
    System.out.println("Default");
  }

  Student(int id) {
    System.out.println("ID=" + id);
  }

  public static void main(String args[]) {
    new Student();
  }
}
```

Output

```
ID=100
Default
```

When `new Student()` runs, the no-argument constructor first calls `this(100)`, which jumps straight into the two — wait, one-argument constructor and prints `ID=100`. Only after that call finishes does control return to print `"Default"`. This is why `ID=100` appears **before** `Default`, even though `System.out.println("Default")` is written first in the no-argument constructor.

Uses of `this`

- Refers to the current object.
- Resolves ambiguity between instance variables and parameters.
- Invokes another constructor (`this()`).
- Passes the current object as an argument.
- Returns the current object.

---

7. Packages and Access Modifiers

Packages

A **package** is a namespace used to organize related classes and interfaces — think of it as a labeled folder that groups similar classes together, the same way you'd have used `java.util.Scanner` in earlier classes.

Advantages:

- Code organization
- Reusability
- Name conflict avoidance
- Access protection

Creating a Package

```
package mypackage;

public class Hello {
  public void display() {
    System.out.println("Welcome");
  }
}
```

Importing a Package

```
import java.util.Scanner;

public class Demo {
  public static void main(String args[]) {

    Scanner sc = new Scanner(System.in);
    System.out.println("Enter Name:");
    String name = sc.nextLine();
    System.out.println(name);

    sc.close();
  }
}
```

You import only the class you actually need (`Scanner`), or you could use `java.util.*` to bring in the whole package — but pulling in exactly what you need keeps things tidy.

Access Modifiers (Accessible in subclasses through inheritance)

Modifier	Same Class	Same Package	Subclass	Other Package
`public`	✅	✅	✅	✅
`protected`	✅	✅	✅	❌ (unless it's a subclass accessing it, via inheritance)
default (no keyword)	✅	✅	❌	❌
`private`	✅	❌	❌	❌

- **`public`** — everyone, everywhere, can access it.
- **`protected`** — visible in the same class, same package, and to subclasses (even in a different package) — this is exactly why `protected` is the go-to choice in inheritance scenarios.
- **default** — no keyword at all; visible only within the same package.
- **`private`** — visible only inside the exact class it's declared in; not even a subclass can touch it directly.

```
class Student {
  private int age;

  public void setAge(int a) {
    age = a;
  }

  public int getAge() {
    return age;
  }
}
```

> **Note for Review:** If you're building an inheritance hierarchy and want a subclass to touch a parent's field directly (without a getter/setter), `protected` is usually the right modifier — `private` would completely block that access, forcing you through getters and setters instead.

8. Encapsulation

What is Encapsulation?

Encapsulation is the process of **binding data (variables) and methods (functions) into a single unit (class)** while restricting direct access to the data. It protects the internal state of an object by making fields **private** and providing **controlled access** through public methods (getters and setters).

Advantages:

- Data hiding
- Improved security
- Better modularity
- Easy maintenance
- Increased flexibility

Example (Student Class)

```
class Student {
  private int rollNo;
  private String name;

  public void setRollNo(int rollNo) {
    this.rollNo = rollNo;
  }

  public int getRollNo() {
    return rollNo;
  }

  public void setName(String name) {
    this.name = name;
  }

  public String getName() {
    return name;
  }

  public static void main(String[] args) {
    Student s = new Student();

    s.setRollNo(101);
    s.setName("Rahul");

    System.out.println("Roll No: " + s.getRollNo());
    System.out.println("Name: " + s.getName());
  }
}
```

Output

```
Roll No: 101
Name: Rahul
```

Notice `rollNo` and `name` are `private` — nobody outside the class can write `s.rollNo = 101` directly. Instead, they must go through the public "front door" methods `setRollNo()` and `getRollNo()`. This is like a bank keeping your balance private but giving you an ATM interface (deposit/withdraw) to interact with it safely.

9. Inheritance

What is Inheritance?

Inheritance allows one class to **acquire the properties and methods** of another class. Imagine a family tree: a child naturally has traits from their parent without needing to relearn them.

Benefits:

- Code reuse
- Easy maintenance
- Extensibility
- Supports polymorphism

Syntax

```
class Parent {
  // members
}

class Child extends Parent {
  // additional members
}
```

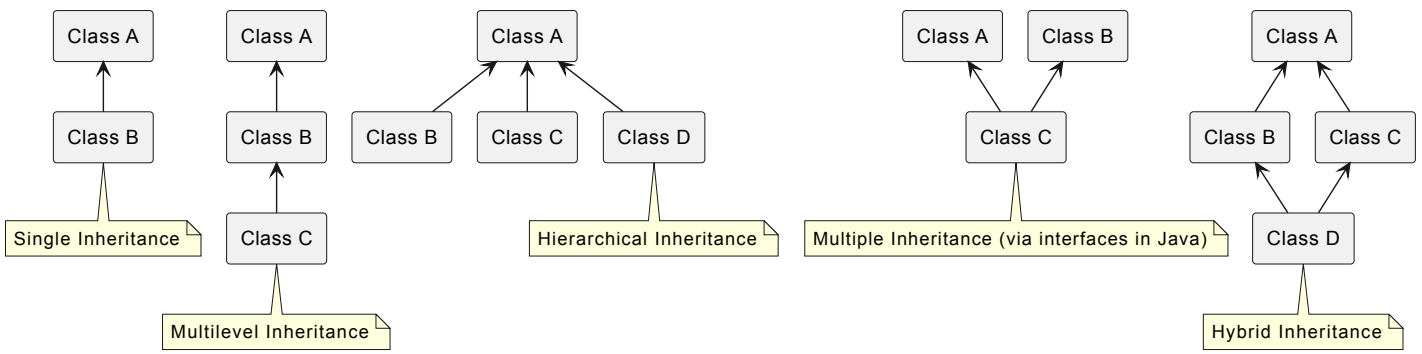
The class being inherited **from** is called the **base class / super class / parent class**. The class doing the inheriting is called the **derived class / sub class / child class**.

```
class Animal {
  void eat() {
    System.out.println("Eating");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Barking");
  }
}
```

Because `Dog` extends `Animal`, a `Dog` object automatically has access to `eat()` too — without writing it again.

Types of Inheritance in Java



Single Inheritance

One parent, one child.

```
class Animal {
  void eat() {
    System.out.println("Animal is eating");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Dog is barking");
  }

  public static void main(String[] args) {
    Dog d = new Dog();
    d.eat();
    d.bark();
  }
}
```

Output

```
Animal is eating
Dog is barking
```

Multilevel Inheritance

A chain: `Animal` → `Dog` → `Puppy`, where each level inherits from the one directly above it.

```
class Animal {
  void eat() {
    System.out.println("Animal Eats");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Dog Barks");
  }
}

class Puppy extends Dog {
  void weep() {
    System.out.println("Puppy Weeps");
  }

  public static void main(String[] args) {
    Puppy p = new Puppy();
    p.eat();
    p.bark();
    p.weep();
  }
}
```

Output

```
Animal Eats
Dog Barks
Puppy Weeps
```

`Puppy` gets `eat()` from `Animal` (via `Dog`) and `bark()` from `Dog` directly, plus its own `weep()` — inheritance stacks up level by level.

Hierarchical Inheritance

One parent, multiple children — each child inherits independently from the same super class.

```
class Animal {
  void eat() {
    System.out.println("Eating");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Barking");
  }
}

class Cat extends Animal {
  void meow() {
    System.out.println("Meowing");
  }

  public static void main(String[] args) {
    Dog d = new Dog();
    Cat c = new Cat();

    d.eat();
    d.bark();

    c.eat();
    c.meow();
  }
}
```

Output

```
Eating
Barking
Eating
Meowing
```

Both `Dog` and `Cat` independently inherit `eat()` from `Animal`, but each brings its own additional behaviour (`bark()` vs `meow()`).

> **Note for Review:** Java does **not** support multiple inheritance of classes (one class extending two classes directly) — to avoid ambiguity (the classic "Diamond Problem"). That's exactly why `interface`s exist: a class can `implement` multiple interfaces to simulate multiple inheritance safely.

10. Polymorphism

What is Polymorphism?

Polymorphism means **"one interface, many forms."** The same method name can perform different tasks depending on the object or the parameters involved.

Types:

1. **Compile-time Polymorphism** (Method Overloading)
2. **Run-time Polymorphism** (Method Overriding)

Compile-Time Polymorphism (Method Overloading)

```
class Calculator {

  int add(int a, int b) {
    return a + b;
  }

  double add(double a, double b) {
    return a + b;
  }

  int add(int a, int b, int c) {
    return a + b + c;
  }

  public static void main(String[] args) {
    Calculator c = new Calculator();

    System.out.println(c.add(5, 4));
    System.out.println(c.add(2.5, 3.5));
    System.out.println(c.add(2, 4, 6));
  }
}
```

Output

```
9
6.0
12
```

This is called **compile-time** because the compiler already knows, just by reading your code, exactly *which* `add` you meant — based purely on the number and type of arguments, before the program even runs.

Run-Time Polymorphism (Method Overriding)

```
class Animal {
  void sound() {
    System.out.println("Animal makes sound");
  }
}

class Dog extends Animal {
  @Override
  void sound() {
    System.out.println("Dog barks");
  }
}

public class Demo {
  public static void main(String[] args) {
    Animal a = new Dog();
    a.sound();
  }
}
```

Output

```
Dog barks
```

Even though the variable `a` is declared as type `Animal`, the actual object stored inside it is a `Dog`. Java waits until **run time** to decide which `sound()` to call, based on the **actual object**, not the declared variable type. This is the heart of run-time polymorphism.

Method Overriding

Method overriding occurs when a subclass provides its own implementation of a method already defined in its superclass.

Rules:

- Method name must be the same.
- Parameters must be identical.
- Return type must be the same or covariant.
- Access level cannot be more restrictive (e.g., you can't override a `public` method with a `private` one).
- Static methods cannot be overridden.

```
class Vehicle {
  void start() {
    System.out.println("Vehicle Starts");
  }
}

class Car extends Vehicle {
  @Override
  void start() {
    System.out.println("Car Starts with Push Button");
  }

  public static void main(String[] args) {
    Car c = new Car();
    c.start();
  }
}
```

Output

```
Car Starts with Push Button
```

Dynamic Method Dispatch

Dynamic Method Dispatch is the mechanism by which a call to an overridden method is resolved **at run time**, based on the actual object being referred to — not on the type of the reference variable.

```
class Animal {
  void sound() {
    System.out.println("Animal Sound");
  }
}

class Dog extends Animal {
  @Override
  void sound() {
    System.out.println("Dog Barks");
  }
}

class Cat extends Animal {
  @Override
  void sound() {
    System.out.println("Cat Meows");
  }
}

public class Test {
  public static void main(String[] args) {
    Animal obj;

    obj = new Dog();
    obj.sound();

    obj = new Cat();
    obj.sound();
  }
}
```

Output

```
Dog Barks
Cat Meows
```

The **same reference variable** `obj` produces **different behaviour** depending on which actual object it currently holds — this is precisely how one interface can wear "many forms."

> **Note for Review:** Overloading is resolved by the **compiler** using the method signature (compile time). Overriding is resolved by the **JVM** using the actual object at run time. If a question ever asks "which happens first, compile or run?" — overloading wins that race.

11. Abstraction

What is Abstraction?

Abstraction is the process of hiding implementation details while exposing only the **essential features** of an object — like a car's steering wheel and pedals (what you interact with) hiding the engine's combustion process (how it actually works).

Java provides abstraction using:

- Abstract classes
- Interfaces

Advantages:

- Simplifies complex systems
- Improves security
- Promotes code reuse
- Supports flexible software design

Abstract Classes

An **abstract class** cannot be instantiated and may contain **both abstract and concrete methods**.

```
abstract class Shape {
  abstract void draw();

  void message() {
    System.out.println("Drawing Shape");
  }
}

class Circle extends Shape {
  @Override
  void draw() {
    System.out.println("Drawing Circle");
  }

  public static void main(String[] args) {
    Circle c = new Circle();
    c.message();
    c.draw();
  }
}
```

Output

```
Drawing Shape
Drawing Circle
```

`Shape` defines `draw()` as abstract (no body at all — a promise, not an implementation) while also giving a ready-made concrete method `message()`. `Circle` is *forced* to fulfil the promise by writing its own `draw()`.

Interface

An **interface** specifies a **contract** that implementing classes must follow.

```
interface Printable {
  void print();
}

class Document implements Printable {
  @Override
  public void print() {
    System.out.println("Printing Document");
  }

  public static void main(String[] args) {
    Document d = new Document();
    d.print();
  }
}
```

Output

```
Printing Document
```

Note the keyword change: a class **extends** another class, but **implements** an interface.

Abstract Class vs Interface

Feature	Abstract Class	Interface
Object creation	Not allowed	Not allowed
Constructors	Yes	No
Instance variables	Yes	Only `public static final` constants
Method implementation	Abstract and concrete methods	Can have abstract, default, and static methods (Java 8+)
Multiple inheritance	No	Yes

> **Note for Review:** Because Java classes can `implement` several interfaces at once, interfaces are the standard workaround for the fact that Java doesn't allow a class to `extend` two classes directly.

---

12. The `super` Keyword

What is `super` ?

The `super` keyword refers to the **immediate parent class object**.

Uses:

- Access superclass variables.
- Invoke superclass methods.
- Invoke superclass constructors.

Accessing Parent Variable

```
class Animal {
  String name = "Animal";
}

class Dog extends Animal {
  String name = "Dog";

  void display() {
    System.out.println(super.name);
    System.out.println(name);
  }

  public static void main(String[] args) {
    Dog d = new Dog();
    d.display();
  }
}
```

Output

```
Animal
Dog
```

Both classes have a field called `name` — normally this would just shadow the parent's version. `super.name` deliberately reaches back into the parent object to grab **its** version, while a plain `name` refers to the child's own copy.

Calling Parent Method

```
class Animal {
  void sound() {
    System.out.println("Animal Sound");
  }
}

class Dog extends Animal {
  @Override
  void sound() {
    super.sound();
    System.out.println("Dog Barks");
  }

  public static void main(String[] args) {
    Dog d = new Dog();
    d.sound();
  }
}
```

Output

```
Animal Sound
Dog Barks
```

Even though `sound()` is overridden, `super.sound()` lets the child **still call the parent's original version** before adding its own behaviour on top — the child doesn't have to throw away the parent's logic entirely.

Calling Parent Constructor

```
class Animal {
  Animal() {
    System.out.println("Animal Constructor");
  }
}

class Dog extends Animal {
  Dog() {
    super();
    System.out.println("Dog Constructor");
  }

  public static void main(String[] args) {
    new Dog();
  }
}
```

Output

```
Animal Constructor
Dog Constructor
```

`super()` explicitly calls the parent's constructor first. In fact, Java inserts an **implicit `super()` call** at the very start of every constructor automatically if you don't write one yourself — this is why a parent always "sets itself up" before the child adds anything on top.

> **Note for Review:** `this()` and `super()` are both special — they must be the **very first statement** inside a constructor, and you can never use both in the same constructor at the same time.

---

13. Summary

After completing this module, you should be able to:

- Apply **encapsulation** using private fields and public getter/setter methods.
- Implement **single, multilevel, and hierarchical inheritance** for code reuse.
- Differentiate between **compile-time and run-time polymorphism**.
- **Override** superclass methods and understand **dynamic method dispatch**.
- Design **abstract classes and interfaces** to achieve abstraction.
- Use the **`super`** keyword to access superclass variables, methods, and constructors.

14. Practice Questions

> Think these through on paper before you touch a keyboard — the goal is to predict the exact output or design, not just recognize the syntax.

1. Suppose `Vehicle` has a `protected` field `speed` and a method `move()`. You create `Car extends Vehicle` and `SportsCar extends Car`, where `SportsCar` overrides `move()` to call `super.move()` first. If you create `Vehicle v = new SportsCar();` and call `v.move();`, trace exactly what executes and in what order — and explain *why* the reference type `Vehicle` doesn't stop the `SportsCar` version from running.

2. A class `Bank` has two constructors: a default one that sets balance to `0`, and a parameterized one that uses `this(0)` internally before printing "Account Opened." If a third constructor is added that takes two parameters and calls a different constructor via `this(...)`, design the constructor chain so that no matter which constructor a user calls, "Account Opened" always prints exactly once. Where would you place the `this()` call, and why?

3. You have an interface `Playable` with a method `play()`, and an abstract class `Instrument` with a concrete method `tune()` and an abstract method `play()`. If `Guitar` both `extends Instrument` and `implements Playable`, is this legal in Java? What rule from the Abstract Class vs Interface comparison table explains your answer, and what would go wrong (or not) if `Playable` also declared a `tune()` method with a `default` implementation?

4. Consider a hierarchical inheritance setup: `Employee` is the superclass, with `Manager` and `Developer` as subclasses, each overriding a `work()` method. If you store all employees in an array of type `Employee[]` and loop through calling `emp.work()` on each, explain how dynamic method dispatch produces different output for each object — and why declaring the array as `Employee[]` (instead of `Object[]`) still allows this variety of behaviour.

5. A `static` variable `totalAccounts` in a `BankAccount` class is incremented inside every constructor. If 5 objects are created using three different overloaded constructors (default, one-argument, two-argument), what will `totalAccounts` equal at the end, and why does it not matter which constructor was used each time? Now suppose you *mistakenly* made `totalAccounts` a normal (non-static) instance variable instead — what changes in behaviour, and why would this defeat the purpose of counting all accounts?